

VM 运行时验收平台

本文面向开发者和测试维护者，说明如何使用可重置 Windows VM 验收唐诗村智能售货机系统机器端运行时。该平台用于缩短反馈循环，验证机器端、daemon、Service API、MQTT、支付、模拟下位机、模拟扫码器、视觉输入和音频行为的主链路。

该平台聚焦已安装 Windows 运行时验收。ISO 制作和场地实机验收分别由独立流程覆盖，机器端业务代码保持与现场一致。测试适配应停在设备或外部服务边界：下位机和扫码器按协议模拟，视觉在 VM 中可使用录制视频源，业务代码仍走与现场一致的运行路径。

1. 运行结构

运行时验收由三个脚本入口组成：

- `scripts/testbed/runtime-testbed-trigger.ts`：开发者入口，负责校验当前提交、推送到测试宿主镜像仓库，并触发运行。
- `scripts/testbed/runtime-testbed-orchestrator.ts`：宿主编排入口，负责重置 VM、准备运行环境、执行指定验收模式并汇总结果。
- `scripts/testbed/business-check-registry.ts`：业务集合注册表，定义可单独执行的验收集合。

测试宿主需要提供一份 JSON 配置文件。配置版本为 `vem-runtime-testbed-host/v1`，至少包含：

- `mirrorPath`：宿主上的 Git 镜像路径。
- `workspaceRoot`：宿主运行工作区根目录。
- `stateRoot`：运行状态和结果输出目录。
- `baselineContract`：Windows 基线 VM 合同文件。
- `hostPrivateAddress`：VM 可访问的宿主内网 IPv4 地址。
- `guestSourcePath`：Windows 客体中的源码路径。
- `visionCoreArtifacts`：宿主本地的核心 Vision 输入，且只能包含 `runtimeArchive` 与 `recordedFixtureArchive` 两项。两项均声明绝对 `hostPath`、小写 SHA-256 和 `byteSize`；运行时归档额外声明 40 位小写 `sourceCommit`。编排器只接受普通文件并在每个 pass 校验摘要和大小、生成快照，再传输到由这两项身份聚合值派生的固定 `C:\ProgramData\VEM\testbed\vision-core\<aggregate>`。
`runtimeArchive` 是已证明的 candidate v3 ZIP；客体逐成员复核其内嵌清单后，只把 `vending-vision` 组装为现有安装器可消费的本地 delivery。`recordedFixtureArchive` 是同一 Vision source commit 生成的独立录播 fixture ZIP。客体启动只使用这两个预置输入，绝不以 Vision 缓存缺失为由查询 GitHub。

`full` 的第一轮还会从已验证的 workflow identity 生成 canonical `vem-runtime-testbed-acceptance-release/v1` 清单，绑定 VEM source commit、Service API 实际健康与 Service API/Admin UI 构建摘要、宿主 owned loopback HTTP 对 Admin UI delivery 入口的逐字节观测、daemon/Machine/WebViewLoader、Vision runtime/fixture 及 Vision V2 contract。第二轮从自身事实重新生成同一清单；任何差异都会使 stability gate 失

败。成功时 compact 根同时保留 `acceptance-release-manifest.json`，`full-workflow-stability-gate.json` 记录其小写 SHA-256。

Windows 基线由测试宿主机从标准 Windows 10 安装介质生成，并以 qcow2 形式保存在宿主机本地。仓库维护生成脚本、验收合同和运行期约束。`baselineContract` 指向宿主机上的小型 JSON 合同或当前发布清单，用来描述本地基线的路径、摘要、显示分辨率、缓存盘和运行时准备状态。

这份基线应等价于一台已完成基础准备的普通 Windows 10 工控机：系统已安装，显示为 1080x1920 竖屏，具备 VM 运行和自动化所需的驱动、OpenSSH、构建缓存盘、默认音频和 Runtime Bootstrap 准备能力。唐诗村智能售货机系统的业务身份、商品、库存、支付、MQTT、扫码器、下位机和视觉行为仍通过真实运行路径或设备边界配置，不固化在基线镜像里。

生成出的 qcow2、缓存盘、ISO 和凭据属于测试宿主机本地状态，留在 Git 之外。

2. 重建 Windows 基线

重建命令在测试宿主机上的仓库检出目录执行。宿主机需要 KVM/QEMU/libvirt、`xorriso`、`qemu-img`、`virt-install`、标准 Windows 10 ISO、VirtIO 驱动 ISO、Actions Runner 压缩包、OpenSSH 登录密钥和 Windows 本地用户密码文件。

先准备宿主机目录。大文件目录使用宿主机磁盘，不使用容器内部文件系统。

```
export VEM_BASELINE_ROOT=/opt/vem/runtime-baseline
export VEM_BASELINE_CONFIG="$VEM_BASELINE_ROOT/config.json"

mkdir -p \
  "$VEM_BASELINE_ROOT/images" \
  "$VEM_BASELINE_ROOT/locks" \
  "$VEM_BASELINE_ROOT/secrets"
```

基线配置使用 `win10-kvm-baseline/v1`。下面是完整字段形态，路径按宿主机实际目录填写。

```
{
  "schemaVersion": "win10-kvm-baseline/v1",
  "host": {
    "address": "testbed-host",
    "libvirtUri": "qemu:///system",
    "lockPath": "/opt/vem/runtime-baseline/locks/win10-runtime-baseline.lock",
    "largeFileRoot": "/opt"
  },
  "vm": {
    "name": "vem-win10-runtime-baseline",
    "networkName": "default",
```

```
"macAddress": "52:54:00:56:45:4d"
},
"storage": {
  "baselinePath": "/opt/vem/runtime-baseline/images/win10-runtime-baseline.qcow2",
  "cacheDiskPath": "/opt/vem/runtime-baseline/images/win10-runtime-cache.qcow2",
  "systemDiskGiB": 80,
  "cacheDiskGiB": 48,
  "minimumFreeGiB": 40
},
"media": {
  "windowsIsoPath": "/opt/vem/assets/windows10.iso",
  "windowsImageIndex": 4,
  "webView2InstallerUri": "https://go.microsoft.com/fwlink/p/?LinkId=2124703",
  "runnerArchivePath": "/opt/vem/assets/actions-runner-win-x64-2.335.1.zip",
  "runnerArchiveSha256":
"eb65c95277af42bcf3778a799c41359d224ba2a67b4de26b7cea1729b09c803d",
  "virtioWinIsoPath": "/opt/vem/assets/virtio-win-stable.iso",
  "spiceGuestToolsInstallerPath": "/opt/vem/assets/spice-guest-tools-0.141.exe"
},
"guest": {
  "administratorPasswordFile": "/opt/vem/runtime-baseline/secrets/administrator-password",
  "authorizedKeysFile": "/opt/vem/runtime-baseline/secrets/administrator-authorized-keys",
  "sshPrivateKeyFile": "/opt/vem/runtime-baseline/secrets/administrator-private-key",
  "sshUser": "VEMKiosk",
  "desktopScalePercent": 100
},
"runner": {
  "url": "https://github.com/YKDZ/vem",
  "name": "testbed-win10-runtime",
  "labels": ["vem-runtime"],
  "registrationTokenProvider": {
    "command": "/opt/vem/runtime-baseline/bin/issue-runner-token",
    "arguments": []
  }
},
"testbed": {
  "reconstructCommand": [
    "/usr/bin/node",
    "{repository}/scripts/testbed/local-testbed-host.ts",
    "reconstruct",
    "--run-id",

```

```

    "{runId}",
    "--libvirt-uri",
    "qemu:///system",
    "--domain-name",
    "vem-win10-runtime-testbed",
    "--overlay",
    "/opt/vem/runtime-testbed/system-overlay.qcow2",
    "--runtime-xml",
    "/opt/vem/runtime-testbed/domain.xml",
    "--filter-name",
    "vem-runtime-testbed-admission",
    "--filter-xml",
    "/opt/vem/runtime-testbed/admission-filter.xml",
    "--host-private-cidr",
    "{hostPrivateAddress}/32",
    "--ssh-host",
    "{guestHost}",
    "--ssh-port",
    "22",
    "--ssh-user",
    "{guestUser}",
    "--identity-file",
    "{identityFile}",
    "--known-hosts-file",
    "{knownHostsFile}",
    "--readiness-timeout-seconds",
    "300",
    "--baseline-system",
    "{systemPath}",
    "--cache-disk",
    "{cachePath}",
    "--domain-xml",
    "{domainXmlPath}"
  ],
  "admitRunnerCommand": [
    "/usr/bin/node",
    "{repository}/scripts/testbed/local-testbed-host.ts",
    "admit",
    "--run-id",
    "{runId}",
    "--libvirt-uri",

```

```

    "qemu:///system",
    "--domain-name",
    "vem-win10-runtime-testbed",
    "--overlay",
    "/opt/vem/runtime-testbed/system-overlay.qcow2",
    "--runtime-xml",
    "/opt/vem/runtime-testbed/domain.xml",
    "--filter-name",
    "vem-runtime-testbed-admission",
    "--filter-xml",
    "/opt/vem/runtime-testbed/admission-filter.xml",
    "--host-private-cidr",
    "{hostPrivateAddress}/32",
    "--ssh-host",
    "{guestHost}",
    "--ssh-port",
    "22",
    "--ssh-user",
    "{guestUser}",
    "--identity-file",
    "{identityFile}",
    "--known-hosts-file",
    "{knownHostsFile}",
    "--readiness-timeout-seconds",
    "300",
    "--guest-input",
    "{guestStagingPath}"
  ],
  "guest": {
    "host": "192.168.122.50",
    "user": "VEMKiosk",
    "identityFile": "/opt/vem/runtime-baseline/secrets/administrator-private-key",
    "knownHostsFile": "/opt/vem/runtime-testbed/known_hosts",
    "stagingPath": "C:\\ProgramData\\VEM\\testbed\\guest-input.json",
    "cacheRoot": "D:\\runtime-cache\\v1"
  }
}
}
}

```

重建前执行预检。该命令只校验配置、宿主机能力、安装介质、磁盘容量和生成计划。

```
node scripts/testbed/kvm-baseline/build-win10-baseline.ts \
  --config "$VEM_BASELINE_CONFIG" \
  | tee "$VEM_BASELINE_ROOT/preflight-plan.json"
```

预检输出的 `execute` 应为 `false`，`profile.display` 应为 `1080x1920`，`profile.vcpus` 和 `profile.memoryMiB` 应符合宿主机配置。

执行重建。该命令会创建一次性构建 VM，完成 Windows 10 无人值守安装，安装运行时依赖，配置 OpenSSH、显示、音频、串口、Runner 和缓存盘，验证通过后发布新的 current manifest。

```
export VEM_COMMIT="$(git rev-parse HEAD)"
export VEM_BASELINE_RESULT="$VEM_BASELINE_ROOT/build-result-$VEM_COMMIT.json"

node scripts/testbed/kvm-baseline/build-win10-baseline.ts \
  --config "$VEM_BASELINE_CONFIG" \
  --source-commit "$VEM_COMMIT" \
  --execute \
  | tee "$VEM_BASELINE_RESULT"
```

重建成功后查看发布结果。

```
jq '{
  promoted,
  publication,
  verification: {
    ok: .verification.ok,
    desktop: .verification.desktop,
    checks: .verification.checks,
    toolchainCommands: (
      .verification.toolchain.commands
      | map({name, version, available})
    )
  },
  profile: {
    vcpus: .profile.vcpus,
    memoryMiB: .profile.memoryMiB,
    display: .profile.display
  }
}' "$VEM_BASELINE_RESULT"
```

```
jq '{
  schemaVersion,
  releaseId,
  destinations,
  artifacts,
  display: .profile.display
}' "$(jq -r '.publication.currentManifestPath' "$VEM_BASELINE_RESULT")"
```

发布清单的 `schemaVersion` 应为 `win10-kvm-baseline-current/v1`。运行时验收宿主配置的 `baselineContract` 指向该 current manifest。

2026-07-26 已在测试宿主机 `forest` 上验证一次独立基线重建。验证根目录为 `/opt/vem/runtime-baseline/doc-verify-20260726`，使用 `/opt/vem/assets/windows10.iso` 和 `/opt/vem/assets/virtio-win-stable.iso`，未覆盖现有运行时基线。预检通过后执行重建并发布：

- current manifest: `/opt/vem/runtime-baseline/doc-verify-20260726/images/win10-baseline-doc-verify.qcow2.current.json`
- release: `release-0d77ab54-5929-4ab1-beff-e51d4e56d547`
- 验证结果: `verification.ok = true`
- 桌面: `1080x1920`、缩放 `100%`、交互用户 `VEMKiosk`
- 运行准备: 显示驱动、WebView2、默认音频、OpenSSH、串口、缓存盘、Actions Runner、Node/pnpm/Turbo/Rust 工具链均通过

上面的配置形态已经按这次验证使用的磁盘规格更新。测试宿主空间充足时，可以把 `systemDiskGiB`、`cacheDiskGiB` 和 `minimumFreeGiB` 调大，预检会在执行前给出容量校验结果。

3. 基本命令

在仓库工作区执行。触发脚本要求当前工作树干净，`--commit` 使用完整 40 位提交 SHA，`--config` 和 `--out` 使用绝对路径。

```
export VEM_COMMIT="$(git rev-parse HEAD)"
export VEM_TESTBED_CONFIG=/abs/path/to/runtime-testbed-host.json
export VEM_TESTBED_RESULT=/abs/path/to/runtime-testbed-result.json
```

运行快速核心集合：

```
node scripts/testbed/runtime-testbed-trigger.ts run \
  --mode fast \
  --commit "$VEM_COMMIT" \
```

```
--config "$VEM_TESTBED_CONFIG" \  
--out "$VEM_TESTBED_RESULT"
```

运行指定业务集合：

```
node scripts/testbed/runtime-testbed-trigger.ts run \  
  --mode fast \  
  --focus sale \  
  --focus scannerPayment \  
  --commit "$VEM_COMMIT" \  
  --config "$VEM_TESTBED_CONFIG" \  
  --out "$VEM_TESTBED_RESULT"
```

运行完整集合：

```
node scripts/testbed/runtime-testbed-trigger.ts run \  
  --mode full \  
  --commit "$VEM_COMMIT" \  
  --config "$VEM_TESTBED_CONFIG" \  
  --out "$VEM_TESTBED_RESULT"
```

查看某次运行状态：

```
node scripts/testbed/runtime-testbed-orchestrator.ts status \  
  --config "$VEM_TESTBED_CONFIG" \  
  --run-id <run-id>
```

清理测试缓存：

```
node scripts/testbed/runtime-testbed-trigger.ts run \  
  --mode clear_cache \  
  --commit "$VEM_COMMIT" \  
  --config "$VEM_TESTBED_CONFIG" \  
  --out "$VEM_TESTBED_RESULT"
```

4. 业务集合

当前注册的业务集合如下：

- `commissioning`：机器认领和初始化。

- `startup`：运行时启动和进程归属。
- `sale`：标准选购、支付、出货主链路。
- `scannerPayment`：扫码器付款码输入链路。
- `visionExperience`：视觉推荐和试衣体验。
- `pickupProtocol`：开门、取货、关门和物理状态提示。
- `presenceAndAudio`：来人/离开、欢迎语音、介绍语音和相关防抖。
- `ipcRecovery`：本地 IPC 恢复。
- `fulfillmentRecovery`：出货失败和恢复。
- `paymentRecovery`：支付失败、订单恢复和库存释放。
- `paymentProvider`：支付提供商可用性，当前用于支付宝真实或沙箱链路专项验证。
- `stockMaintenance`：机器端库存维护和补货。
- `hardwareLifecycle`：硬件绑定和设备生命周期。
- `localOperations`：机器端维护入口与本地操作。
- `environmentControl`：空调、出风口和风速控制。

`fast` 不带 `--focus` 时只运行核心集合：`sale` 与 `stockMaintenance`。`full` 运行所有默认完整集合。

`paymentProvider` 不包含在默认完整集合中，需要用 `fast --focus paymentProvider` 显式执行，避免第三方支付环境波动阻塞普通业务回归。

5. 结果与证据

触发命令会把运行结果写入 `--out` 指定的 JSON 文件。优先查看其中的整体状态、`runId`、失败集合和证据路径；完整日志、截图和报告进入运行结果目录。

分析失败时按以下顺序处理：

1. 先判断是宿主机调度、VM 启动、构建部署、外部服务，还是业务断言失败。
2. 若是页面或业务状态观测过早，优先改为等待明确业务状态，不直接增加固定等待。
3. 若测试平台缺少可观测性，优先在业务层补充真实可用的状态或日志，再让测试读取该状态。
4. 若多个集合互相污染状态，应在集合开始前做轻量业务恢复，例如返回目录、释放活动交易、补足目标货道库存、重开串口会话。

6. 维护规则

- VM 测试和现场部署应共用同一条机器端运行路径，不维护测试专用业务分支。
- 不按固定 COM 号绑定设备；扫码器和下位机按 USB/PnP 身份与协议边界识别。
- 不把第三方支付沙箱的高频异常扩大成复杂业务兜底；生产代码应能给用户显示语义化错误并释放业务状态。
- 一轮 `fast` 或 `full` 超过 10 分钟时，应查看各阶段耗时并优先优化反馈循环，不盲目延长超时。
- 文档和 ADR 内容不作为自动化测试断言对象；测试应验证代码、接口、运行状态和用户可见行为。